Лабораторна робота №9. Навігація в React Native

Мета роботи

Навчитися налаштовувати React Navigation, реєструвати типізовані маршрути, переходити між екранами та повертатися назад. Порівняти Native Stack, нижні вкладки й бічне меню Drawer. Зберегти узгоджені кольори, роботу форми та безпечні відступи застосунку.

Що має працювати наприкінці

Продовжте застосунок **Little Lemon** із №6–8. Підготуйте три окремі варіанти навігації, збережені у різних файлах:

1. **Native Stack:** початковий екран `Login`, перехід до `Welcome` після локальної перевірки форми та повернення через стандартну кнопку заголовка.
2. **Bottom Tabs:** два навчальні розділи `Login` і `Welcome`, перемикання вкладок, власні підписи та іконки.
3. **Drawer:** ті самі два розділи в бічному меню, яке відкривається кнопкою й жестом на підтримуваній мобільній платформі.

Основні завдання виконують послідовно, але попередні варіанти не видаляють. Одночасно активний один навігатор. Об’єднання всіх трьох у складне дерево не є обов’язковим.

Форма залишається навчальною. Перевірка формату email і непорожнього пароля не підтверджує особу користувача. Для дослідження вкладок і Drawer екран привітання можна відкрити без форми. Не додавайте серверну авторизацію, токени чи зберігання паролів. Використовуйте вигадані дані.

Підготовка

Збережіть проєкт із №8 у Git або зробіть резервну копію без `node_modules`. Відкрийте його папку в терміналі. Приклади розраховані на Expo-проєкт з `App.tsx`, створений через `blank-typescript`, а не на структуру Expo Router з `_layout.tsx`.

Установіть пакети React Navigation **7.x**:

```
npm install @react-navigation/native@^7 @react-navigation/native-stack@^7
npm install @react-navigation/bottom-tabs@^7 @react-navigation/drawer@^7
npx expo install react-native-screens react-native-safe-area-context
npx expo install react-native-gesture-handler @expo/vector-icons
```

Для сучасного Expo SDK з Reanimated 4 також виконайте:

```
npx expo install react-native-reanimated react-native-worklets
```

Якщо SDK вашого проєкту використовує **Reanimated 3**, замість останньої команди встановіть тільки `npx expo install react-native-reanimated`, без окремого `react-native-worklets`. `npx expo install` підбирає сумісну версію, а `npx expo install --check` перевіряє залежності поточного SDK. Не змішуйте різні покоління Reanimated і не обходьте конфлікти через `--force`.

Основа React Navigation, Native Stack, вкладки й Drawer — різні пакети. Native Stack сам по собі не потребує Reanimated; додаткові бібліотеки жестів та анімації тут установлюються для Drawer.

У сучасному Expo потрібний плагін Reanimated/Worklets налаштовує `babel-preset-expo`. Не додавайте другий старий плагін у Babel без потреби. Власну development build після додавання нативних залежностей треба перебудувати. Для Expo Go використовуйте версію, сумісну з SDK проєкту.

Після встановлення:

```
npx expo install --check
npx expo start
```

Збережіть `package.json` і `package-lock.json`. Не оновлюйте всі попередні залежності тільки заради навігації.

Теоретичні відомості

Маршрут, екран і навігатор

Компонент екрана повертає його вміст. **Маршрут** пов'язує назву, компонент і параметри. **Навігатор** керує переходами між зареєстрованими маршрутами.

```
<Stack.Screen name="Welcome" component={WelcomeRoute} />
```

`Welcome` — назва для виклику переходу; `WelcomeRoute` — функція компонента, яку навігатор викличе. Видимий заголовок задають окремо через `options={{ title: 'Привітання' }}`.

У динамічному API один кореневий `NavigationContainer` містить активний навігатор. Не створюйте новий контейнер для кожного екрана. Сам навігатор і компоненти маршрутів оголошуйте на рівні модуля, поза `App`, щоб звичайне оновлення стану не створювало їх заново.

Чим відрізняються три варіанти

Stack зберігає послідовність відкритих екранів. Після `Login`, потім `Welcome` повернення прибирає `Welcome` і показує попередній `Login`.

Вкладки та Drawer перемикають доступні розділи. Звідси важливий наслідок: `Welcome` можна вперше відкрити прямо через вкладку чи пункт меню, без email. Його компонент має вміти показати гостя. Для перемикання розділів використовують `navigate`, а не стекові `push` чи `popToTop`.

У реальному застосунку сценарій входу зазвичай відокремлюють від основних розділів. Тут `Login` і `Welcome` у вкладках потрібні для порівняння механізмів, а не для захисту даних.

Типи параметрів

```
type StackParamList = {
  Login: undefined;
  Welcome: { email: string };
};
```

`undefined` означає, що маршрут не потребує параметрів. На `Welcome` у стековому сценарії передається об'єкт з email. TypeScript перевірятиме назву маршруту, наявність параметра та його тип.

Для вкладок контракт інший:

```
type TabParamList = {
  Login: undefined;
  Welcome: { email?: string } | undefined;
};
```

Весь об'єкт може бути відсутній, а його поле `email` також необов'язкове. Запис `route.params?.email ?? 'Гість'` коректно обробляє обидва випадки. Типи описують контракт вашого коду, але не перетворюють email на перевірений обліковий запис.

Компонент інтерфейсу й навігаційна оболонка

У №8 `LoginScreen` отримував `colors` і `onContinue`, а `WelcomeScreen` — `colors`, `email` та `onExit`. Збережіть такий поділ. Не намагайтеся відразу використати ці функції як `component={LoginScreen}`: навігатор автоматично передає `navigation` і `route`, але не ваші довільні props.

Створіть невелику оболонку маршруту. Вона отримує службові props навігатора та передає екрану інтерфейсу потрібні значення і callback. Це дозволить використати ту саму форму з різними навігаторами, не підмінюючи типи через `any`.

Що відбувається зі станом

При відкритті `Welcome` поверх `Login` форма в стеку зазвичай залишається змонтованою. Email може зберегтися після повернення. Пароль потрібно явно очищати перед переходом, як у №8. При повному перезапуску застосунку локальний стан зникає.

Зміна системної теми повинна оновлювати кольори, а не створювати нову історію. Не задавайте навігатору `key`, що змінюється разом із темою.

Виконання роботи

Крок 1. Підготуйте файли й кольори

Рекомендована структура додаткової частини проєкту:

```
App.tsx
navigation/
  types.ts
  StackFlow.tsx
  TabsFlow.tsx
  DrawerFlow.tsx
screens/
  LoginScreen.tsx
  WelcomeScreen.tsx
theme/
  palette.ts
  useAppColors.ts
```

Зображення, меню та інші файли з попередніх робіт залиште на місці. В `theme/palette.ts` уже мають бути `lightColors`, `darkColors` і тип `Palette` із №8.

Створіть `theme/useAppColors.ts`:

```tsx
import { useColorScheme } from 'react-native';
import { darkColors, lightColors } from './palette';

export function useAppColors() {
  return useColorScheme() === 'dark' ? darkColors : lightColors;
}
```

Це власний хук: його назва починається з `use`, а всередині він викликає системний хук. Кожна оболонка маршруту отримуватиме актуальну палітру без копіювання її у додатковий стан.

У `WelcomeScreen` замініть категоричний напис про перевірку даних нейтральним: «Навчальний екран привітання для ...». Він підходить і для форми, і для прямого відкриття гостем. Логотип і опис закладу збережіть.

У `App.tsx` приберіть перемикання екранів через `demoEmail` та тернарний оператор із №8. Відтепер поточний маршрут зберігатиме навігатор. Локальні стани полів у `LoginScreen` залиште.

Завдання 1. Налаштуйте маршрути Native Stack

1.1. Опишіть параметри

У `navigation/types.ts` додайте:

```tsx
export type StackParamList = {
  Login: undefined;
  Welcome: { email: string };
```

```
};

export type TabParamList = {
  Login: undefined;
  Welcome: { email?: string } | undefined;
};

export type DrawerParamList = {
  Login: undefined;
  Welcome: { email?: string } | undefined;
};
```

Це три окремі контракти. Назви двох маршрутів збігаються, але вони належать різним варіантам навігації. Для параметрів використовуйте `type`, без розширення `ParamListBase`.

1.2. Створіть оболонки та стек

У `navigation/StackFlow.tsx` розмістіть:

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import type { NativeStackScreenProps } from '@react-navigation/native-stack';
import LoginScreen from '../screens/LoginScreen';
import WelcomeScreen from '../screens/WelcomeScreen';
import { useAppColors } from '../theme/useAppColors';
import type { StackParamList } from './types';

const Stack = createNativeStackNavigator<StackParamList>();
type LoginProps = NativeStackScreenProps<StackParamList, 'Login'>;
type WelcomeProps = NativeStackScreenProps<StackParamList, 'Welcome'>;

function LoginRoute({ navigation }: LoginProps) {
  const colors = useAppColors();
  return <LoginScreen colors={colors} onContinue={(email) =>
    navigation.navigate('Welcome', { email })
  } />;
}

function WelcomeRoute({ navigation, route }: WelcomeProps) {
  const colors = useAppColors();
  return <WelcomeScreen
    colors={colors}
    email={route.params.email}
    onExit={() => navigation.goBack()}
  />;
}

export default function StackFlow() {
  return (
    <Stack.Navigator initialRouteName="Login">
      <Stack.Screen name="Login" component={LoginRoute}
        options={{ title: 'Навчальна форма' }} />
      <Stack.Screen name="Welcome" component={WelcomeRoute}
        options={{ title: 'Привітання' }} />
    </Stack.Navigator>
```

```
  );
}
```

`NativeStackScreenProps` описує `navigation` та `route` відповідно до вибраного маршруту. На `Welcome` параметр email обов'язковий, тому тут допустиме пряме звернення `route.params.email`.

Якщо власні екрани мають іменовані експорти, виправте імпорти у фігурних дужках. Не змінюйте навмання всі експорти проєкту. Якщо ви інакше назвали callback у №8, узгодьте обидві сторони передачі властивості.

1.3. Додайте єдиний корінь навігації

У `App.tsx` використайте такий каркас:

```
import { useColorScheme } from 'react-native';
import { NavigationContainer, DarkTheme, DefaultTheme } from '@react-navigation/native';
import { GestureHandlerRootView } from 'react-native-gesture-handler';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import StackFlow from './navigation/StackFlow';

// Після створення інших варіантів додайте їхні імпорти.
const ActiveNavigator = StackFlow;

export default function App() {
  const isDark = useColorScheme() === 'dark';
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <SafeAreaProvider>
        <NavigationContainer theme={isDark ? DarkTheme : DefaultTheme}>
          <ActiveNavigator />
        </NavigationContainer>
      </SafeAreaProvider>
    </GestureHandlerRootView>
  );
}
```

Для Native Stack оболонка жестів зайва, але тут вона одразу готує корінь і для майбутнього Drawer. `ActiveNavigator` указує на функцію компонента, тому в JSX записуємо `<ActiveNavigator />`. Історія належить навігатору, а не окремому `useState` у `App`.

Стандартні заголовки й нижні навігаційні панелі вже враховують системні межі. Не обгортайте весь `NavigationContainer` у старий `SafeAreaView` з додатковими верхнім і нижнім відступами. Власний вміст екрана за потреби має свою безпечну область. При видимому стандартному заголовку зазвичай не потрібно додавати верхній відступ удруге; при вкладках також не дублюйте нижній відступ їхньої панелі.

Власну шапку Little Lemon можна залишити всередині вмісту екрана, але не дублюйте однаковий великий заголовок без потреби. Підвал зберігайте у структурі екрана так, щоб він не перекривав кнопки чи навігаційні панелі. Перевірте прокручування й клавіатуру після зміни оболонки.

Запустіть застосунок. Початковим має бути `Login` із заголовком «Навчальна форма». Початкова реєстрація двох маршрутів ще не означає, що потрібно одночасно показувати обидва екрани.

Завдання 2. Перейдіть до привітання та поверніться назад

У `LoginScreen` збережіть обробник перевірки з №8. Його успішна гілка має:

1. Прибрати повідомлення про помилку.
2. Закрити клавіатуру.
3. Очистити пароль.
4. Викликати `onContinue(normalizedEmail)`.

Тепер цей callback виконує `navigation.navigate('Welcome', { email })` в оболонці `LoginRoute`. Компонент форми не повинен додатково створювати власний `NavigationContainer` або перемикати `App` через колишній setter `demoEmail`.

Перевірте:

- Порожня форма та некоректний email не виконують переходу.
- Коректний вигаданий email і непорожній навчальний пароль відкривають `Welcome`.
- У привітанні показується email із параметрів, але не пароль.
- Стандартна кнопка «Назад» у заголовку повертає до форми.
- Власна кнопка `onExit` виконує таке саме повернення.
- На Android системна кнопка або жест повернення також працює відповідно до історії.

Не приховуйте стандартний заголовок у цьому варіанті: саме він демонструє автоматичну кнопку повернення. На початковому `Login` її немає, тому що перед ним немає іншого запису стеку.

Після повернення email у формі може залишитися: компонент не обов'язково демонтувався. Пароль повинен бути порожнім завдяки явному очищенню. Для порівняння тимчасово використайте `replace` замість `navigate`, дослідіть історію, потім поверніть `navigate`: у результаті обов'язково має працювати повернення до форми.

Завдання 3. Створіть нижні вкладки

3.1. Підготуйте окремі оболонки

Створіть `navigation/TabsFlow.tsx`. Додайте імпорти тих самих двох екранів і хука палітри, а також:

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import type { BottomTabScreenProps } from '@react-navigation/bottom-tabs';
import Ionicons from '@expo/vector-icons/Ionicons';
import type { ComponentProps } from 'react';
import type { TabParamList } from './types';

const Tab = createBottomTabNavigator<TabParamList>();
type LoginProps = BottomTabScreenProps<TabParamList, 'Login'>;
type WelcomeProps = BottomTabScreenProps<TabParamList, 'Welcome'>;
type IconName = ComponentProps<typeof Ionicons>['name'];
```

`LoginRoute` побудуйте за зразком стекового, але з типом `BottomTabScreenProps`. У `onContinue` виконайте той самий перехід із email. `WelcomeRoute` для вкладок відрізняється:

```
function WelcomeRoute({ navigation, route }: WelcomeProps) {
  const colors = useAppColors();
  return <WelcomeScreen
    colors={colors}
    email={route.params?.email ?? 'Гість'}
    onExit={() => navigation.navigate('Login')}
  />;
}
```

Тут не можна вимагати наявності `route.params`: вкладка відкривається й прямим натисканням. Для повернення до форми явно вибираємо `Login`, оскільки це перемикання розділів, а не зняття верхнього екрана стеку.

3.2. Налаштуйте вкладки й іконки

Після оболонок додайте:

```
export default function TabsFlow() {
  return (
    <Tab.Navigator initialRouteName="Login"
      screenOptions={({ route }) => ({
        tabBarActiveTintColor: '#175CD3',
        tabBarIcon: ({ color, size, focused }) => {
          const name: IconName = route.name === 'Login'
            ? (focused ? 'log-in' : 'log-in-outline')
            : (focused ? 'home' : 'home-outline');
          return <Ionicons name={name} color={color} size={size} />;
        },
      })}>
      <Tab.Screen name="Login" component={LoginRoute}
        options={{ title: 'Форма', tabBarLabel: 'Форма' }} />
      <Tab.Screen name="Welcome" component={WelcomeRoute}
        options={{ title: 'Привітання', tabBarLabel: 'Привітання' }} />
    </Tab.Navigator>
  );
}
```

`focused` вибирає активний чи контурний варіант іконки. `color` і `size` надходять від навігатора. Тип `IconName` допомагає уникнути неіснуючої назви іконки. Решту кольорів навігаційна панель отримує з теми контейнера.

У `App.tsx` додайте імпорт `TabsFlow` та замініть лише `const ActiveNavigator = StackFlow` на `const ActiveNavigator = TabsFlow`. Залиште кореневий контейнер і збережений файл стеку. Після повного перезавантаження перевірте пряме відкриття `Welcome`: має бути гостьове привітання без помилки. Потім перейдіть через форму й перевірте email.

Параметри маршруту можуть зберегти попередній email при наступних перемиканнях вкладок. Це не стан авторизованого користувача. Якщо хочете повернути гостьовий варіант, повністю перезавантажте приклад або окремо реалізуйте скидання необов'язкових параметрів.

Завдання 4. Реалізуйте Drawer

Створіть `navigation/DrawerFlow.tsx`. Потрібен саме `createDrawerNavigator`, а не повторне створення нижніх вкладок.

Імпорти й типи:

```
import { createDrawerNavigator } from '@react-navigation/drawer';
import type { DrawerScreenProps } from '@react-navigation/drawer';
import LoginScreen from '../screens/LoginScreen';
import WelcomeScreen from '../screens/WelcomeScreen';
import { useAppColors } from '../theme/useAppColors';
import type { DrawerParamList } from './types';

const Drawer = createDrawerNavigator<DrawerParamList>();
type LoginProps = DrawerScreenProps<DrawerParamList, 'Login'>;
type WelcomeProps = DrawerScreenProps<DrawerParamList, 'Welcome'>;
```

Оголосіть `LoginRoute` і `WelcomeRoute` за зразком вкладок, але з цими Drawer-типами. Збережіть перевірку `route.params?.email ?? 'Гість'`. Callback форми переходить на `Welcome`, а кнопка повернення явно вибирає `Login`.

Нижче розмістіть навігатор:

```
export default function DrawerFlow() {
  return (
    <Drawer.Navigator initialRouteName="Login"
      screenOptions={{ drawerPosition: 'left' }}>
      <Drawer.Screen name="Login" component={LoginRoute}
        options={{ title: 'Навчальна форма', drawerLabel: 'Форма' }} />
      <Drawer.Screen name="Welcome" component={WelcomeRoute}
        options={{ title: 'Привітання', drawerLabel: 'Привітання' }} />
    </Drawer.Navigator>
  );
}
```

У `App.tsx` імпортуйте `DrawerFlow` та виберіть його через `ActiveNavigator`. `GestureHandlerRootView` уже є в корені; повторювати його та `NavigationContainer` у цьому файлі не треба.

Перевірте відкривання меню стандартною кнопкою заголовка, вибір обох пунктів і жест відкривання на Android/iOS. Тимчасово змініть `drawerPosition` на `'right'`, дослідіть результат, потім залиште обраний варіант. Для керування з власної кнопки екрана доступний `navigation.openDrawer()`.

У цьому варіанті немає панелі нижніх вкладок. Якщо вона лишилася, перевірте, який компонент обрано в `ActiveNavigator` і чи не вклали ви Drawer у старий Tabs ненавмисно.

Додаткове завдання. Параметри деталей або вкладення

Оберіть один варіант:

1. Додайте до стеку `Menu` і `Details: { itemId: string }`. Відкривайте деталі страви за стабільним `id` із №7. Покажіть зрозумілий стан для невідомого ідентифікатора. Передавайте `id`, а не копію всієї страви.
2. Розмістіть вкладки як маршрут `Main` у стеку. Залиште один контейнер, приховайте дубль заголовка через `headerShown: false` для `Main` і типізуйте вкладені параметри через `NavigatorScreenParams`.

Для обох варіантів пояснення та приклади є у конспекті теми 9. У базовій роботі не потрібно робити всі можливі комбінації навігаторів чи справжній сценарій авторизації.

Перевірка результату

Виконайте:

```
npx tsc --noEmit
npx expo install --check
```

Команда TypeScript перевіряє типи без створення окремих JS-файлів. Вона повинна перевірити всі три збережені модулі, навіть коли активний лише один. Виправте причину помилки замість використання `any` або `@ts-ignore`.

Проведіть перевірки для кожного варіанта після повного перезавантаження:

Варіант	Що перевірити
Stack	Початковий `Login`; некоректна форма не переходить; коректна відкриває `Welcome`; стандартне повернення працює.
Tabs	Обидві вкладки доступні, мають підписи й іконки; пряме відкриття `Welcome` не викликає помилки; callback передає email.
Drawer	Меню відкривається кнопкою, містить правильні пункти й перемикає екрани; жест працює на підтримуваному пристрої.
Усі три	Пароль очищується й не передається в маршрут; логотип і опис збережені; поля та повідомлення читаються в обох темах.
Усі три	Клавіатура не приховує потрібну кнопку; зайві safe-area-відступи й дубльовані заголовки відсутні.

Зміна системної теми не повинна скидати історію сама по собі. Невідомий маршрут або неправильний регістр у його назві — помилка, яку треба усунути, а не спосіб переходу.

Для передачі роботи підготуйте вихідні файли, `package.json`, `package-lock.json` і короткий `README` з командами запуску та способом вибору `ActiveNavigator`. Додайте знімки початкового екрана, переходу в стеку, нижніх вкладок і відкритого Drawer. `node_modules` не додавайте.

Приклад для ознайомлення

У конспекті теми 9 є повний мінімальний `App.tsx` **«Путівник містом»**. Він демонструє Native Stack, параметр `itemId`, відкривання ще одного екземпляра деталей та модальний маршрут довідки. Його можна запустити в окремому `blank-typescript` проєкті після встановлення зазначених залежностей.

Приклад тематично відрізняється від Little Lemon. Використовуйте його для дослідження історії та типів, не замінюйте ним свою форму й інші результати попередніх робіт.

Типові помилки

Проблема	Як виправити
Нативний стек імпортовано з `@react-navigation/stack`	Використайте `@react-navigation/native-stack`.
`LoginScreen` вимагає `colors` і callback, а навігатор їх не передає	Зареєструйте оболонку `LoginRoute`, яка передасть потрібні props.
`Welcome` падає при прямому відкритті вкладки	Зробіть параметри необов'язковими та використайте `?.` і `??`.
У Drawer знову нижні вкладки	Використайте `createDrawerNavigator` і перевірте `ActiveNavigator`.
Не обробляється `NAVIGATE`	Перевірте назву, регістр і реєстрацію маршруту в активному навігаторі.
Після додавання Drawer немає нативного модуля	Перевірте сумісність пакетів та перебудуйте власну development build.
Зайвий проміжок над заголовком чи під вкладками	Приберіть дублювання safe areas навігатора й зовнішньої оболонки.
Повернення не відкриває форму	У стековому варіанті має бути `navigate`, а не `replace`; не приховуйте стандартний заголовок.
Поля очищуються від зміни кольорів	Не змінюйте `key` навігатора за темою і не оголошуйте його всередині `App`.

Захист роботи

Захист усний, із демонстрацією трьох варіантів. Покажіть локальну перевірку форми, передавання email, стандартне повернення в стеку, пряме відкриття гостьового привітання у вкладках і Drawer. Поясніть структуру файлів, типи маршрутів, призначення оболонок і поведінку локального стану.

Будьте готові відповісти на контрольні питання та внести мінімальні зміни: змінити заголовок без перейменування маршруту, замінити іконку вкладки, змінити бік Drawer, додати необов'язковий параметр або скоригувати гостьовий напис. Поясніть, які зміни впливають на історію, а які — лише на вигляд.

Контрольні питання

1. Чим відрізняються компонент екрана, маршрут і екземпляр у стеку?
2. Для чого потрібні `NavigationContainer`, `Navigator` і `Screen`?
3. Які props автоматично передає навігатор? Навіщо створено оболонки маршрутів?
4. Що означає `undefined` у списку параметрів?
5. Чому в стеку email обов'язковий, а у вкладках і Drawer — ні?
6. Яка різниця між `navigate`, `push`, `goBack` і `replace`?
7. Чому email форми може зберегтися після повернення, а пароль має бути очищено?
8. Чому вкладка з формою входу не забезпечує авторизацію?
9. Як `focused`, `color` і `size` використовуються для іконки?
10. Які залежності та коренева оболонка потрібні Drawer?
11. Чому не можна змінювати `key` навігатора разом із темою без очікуваного скидання історії?
12. Чому перезапуск Metro не завжди замінює перебудування нативного застосунку?

Корисні джерела

- React Navigation: початок роботи.
- React Navigation: Native Stack.
- React Navigation: TypeScript.
- React Navigation: параметри маршрутів.
- React Navigation: нижні вкладки.
- React Navigation: Drawer.
- Expo: Reanimated і Worklets.
- React Navigation: безпечні відступи.